🔥 Smart “Fire” Detection & Extinguish System

A helmet that watches, a turret that obeys, and a flame that has nowhere to hide.

The Story in Motion

Put on the helmet, turn your head, and a pan-tilt turret across the room turns with you, left and right, up and down, as if it shared your gaze. A gyroscope (MPU6050) on the helmet reads every tilt and turn and sends it wirelessly to the base. The servos copy it instantly.

Then the camera finds it: **FIRE**.

1. **The alarm sounds.** The buzzer beeps and tells everyone something is wrong.
2. **The system holds its breath.** For two seconds the servos freeze and lock onto the target.
3. **The strike.** The mini pump spits water at the marked spot for six full seconds.
4. **The calm after.** The system lets go and returns to normal mode, ready for the next flame.

Prefer to take control yourself? A **joystick** steers the pan-tilt, and **two push buttons** command the pump by hand. Man and machine share the same weapon.

Two Systems, One Mission

🪖 The Helmet System (the eyes and the instinct)

Component	Role
ESP32 (Transmitter)	Sends signals to the base over **ESP-NOW**
MPU6050 Gyroscope	Reads head movement and steers the pan-tilt
OV2640 Camera	Looks for the word **"FIRE"** and marks the target with a red thread
Buck Converter	Steps 12V down to 5V for the ESP32

🛡️ The Base System (the muscle)

Component	Role
ESP32-S3 (Receiver)	Pairs with the helmet ESP32 by MAC address (ESP-NOW)
Buck Converter ×2	12V → 5V for the ESP32, and 12V → 6V for the two servo motors
Pan-Tilt with Servos	Aims left-right and up-down
12V Mini Pump	Delivers the six-second spray once fire is confirmed
MOSFET (IRLZ44N)	Controls the pump and its pressure
Buzzer	Sounds the alert when fire is identified
Push Buttons	Manual pump control (one switches between high and low pressure)
Joystick Controllers	Manual control of the pan-tilt servos

⚙️ Design Parameters: The Challenge That Shaped the System

This project began with a single purpose: **detect fire and extinguish it.**

My first plan was to let the ESP32-CAM spot flames through **image processing** alone. In testing, it failed in a way I hadn't expected. The camera couldn't tell flame from anything else that looked like it, and **it treated every yellow object as fire.** A yellow bottle, a yellow shirt, a yellow notebook: all of them triggered the alarm.

That was the moment I realized: **"If I keep this, my whole project can fail."**

So I changed the approach. Instead of trusting colour, I taught the camera to **read**. Now, when it sees the letters **"FIRE"**, the system treats it as a confirmed fire and runs the full response. It was a hard lesson in engineering: when a clever solution is unreliable, a smarter, simpler one is better.

Detect. Alert. Aim. Extinguish. Repeat. 🚒